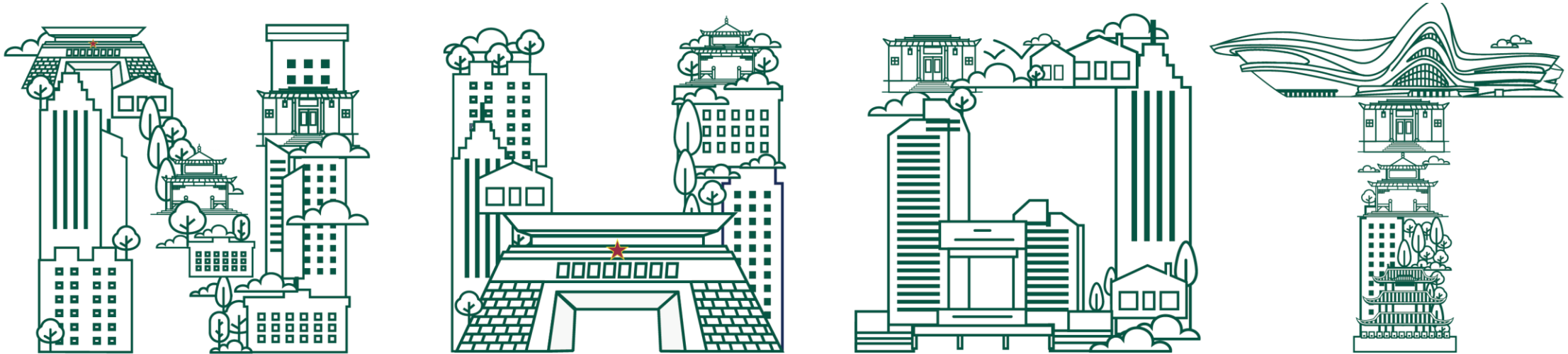
One Size Does Fit All: Exploring Model Fusion for Software Engineering Tasks

Yinggang Qiu, Yihao Qin, Mingyang Geng,
Shangwen Wang✉, Dezun Dong✉

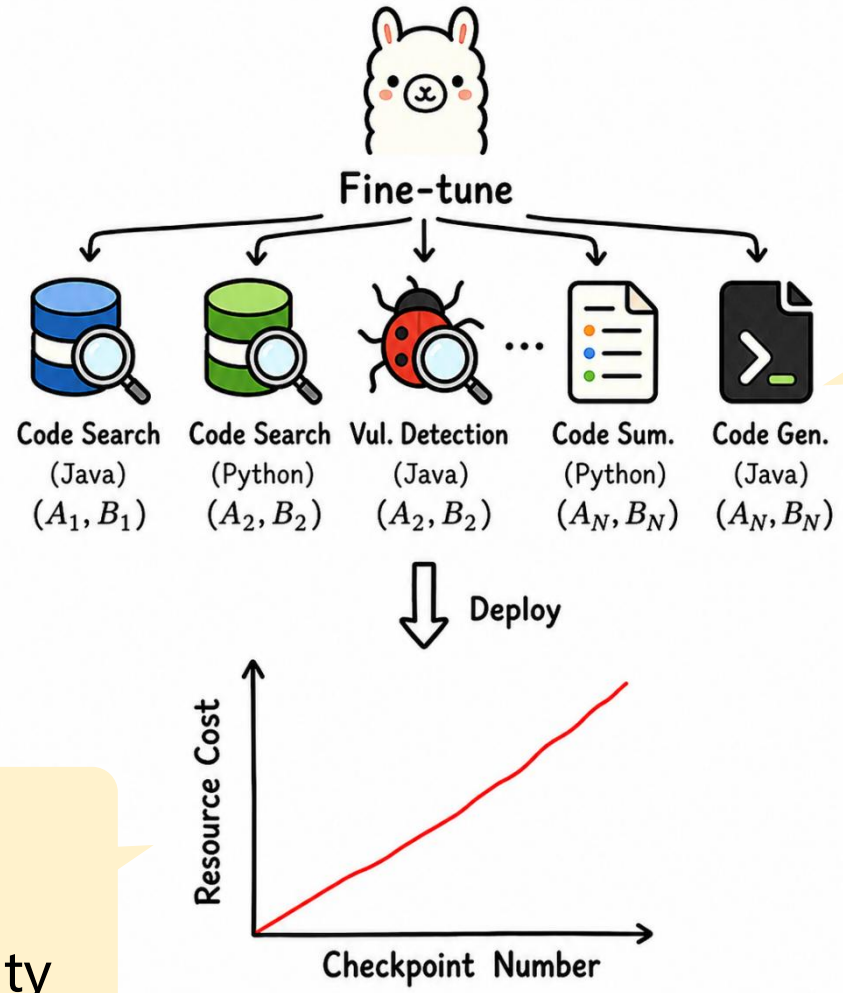
College of Computer Science and Technology
National University of Defense Technology, China

2026-7-8, Wednesday, @FSE 2026



1. Background

➢ Fine-tuning Paradigm

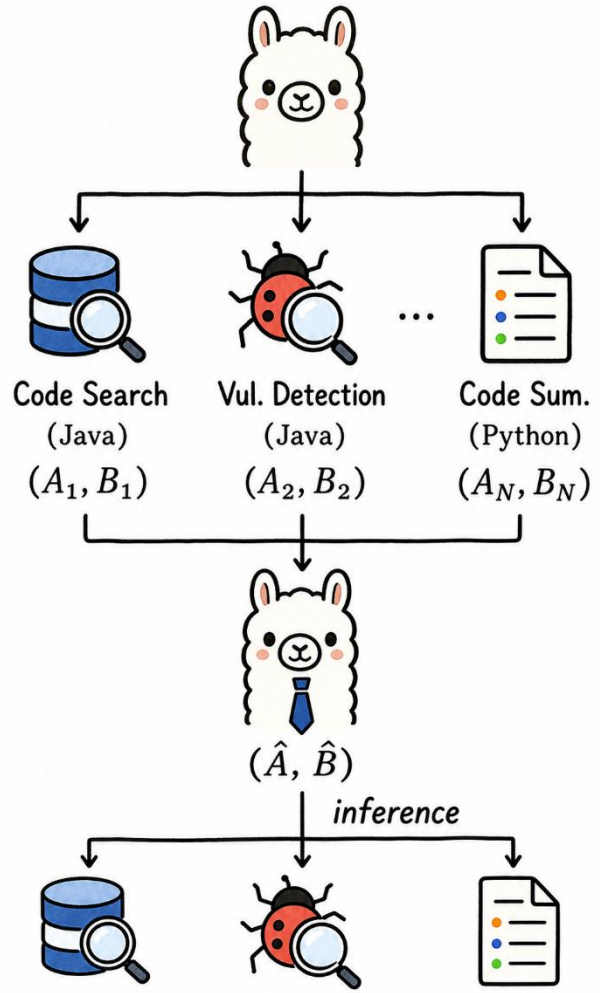


- One checkpoint is fine-tuned for each SE task and PL.

- Difficulties in checkpoint management
- High deployment complexity

1. Background

Model Fusion has achieved strong results in CV and NLP tasks.
Research Goal: Can model fusion enable one unified model to efficiently support multiple SE tasks and programming languages?



1. Pre-trained Model

- Use **CodeLlama-7B** as the base model for fine-tuning across all tasks.

2. Fine-tuning

- For each tasks, train a separate **low-rank adapters (LoRA)** checkpoint.

3. Model Fusion

- Merge multiple LoRA checkpoint into **a single fused parameter set** using a model fusion algorithm.

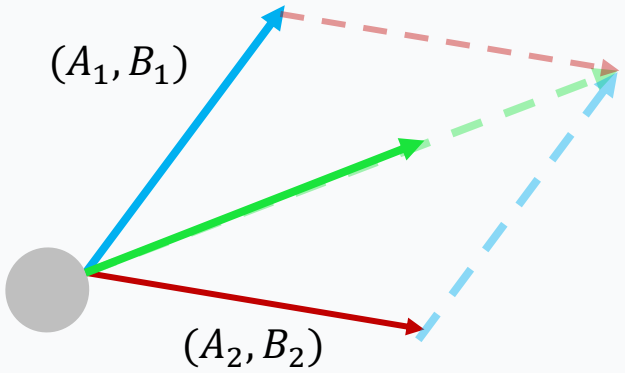
4. Inference

- Load the fused parameters back into the base model and evaluate the fusion performance.

2. Study Design

➢ Selected Model Fusion Approach

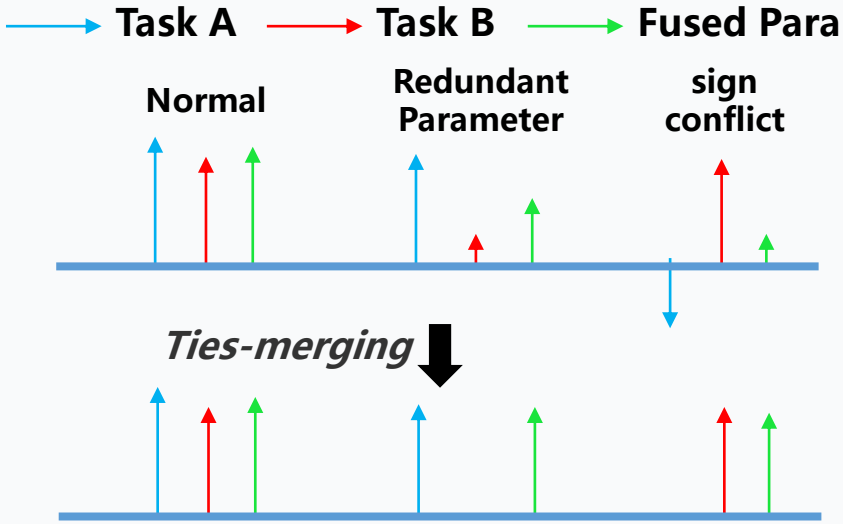
Task Arithmetic [1]



$$\hat{A} = \alpha \sum_{i=0}^{N} A_i\,, \qquad \hat{B} = \alpha \sum_{i=0}^{N} B_i$$

Treat fine-tuned parameters as **vectors**.

Ties-merging [2]



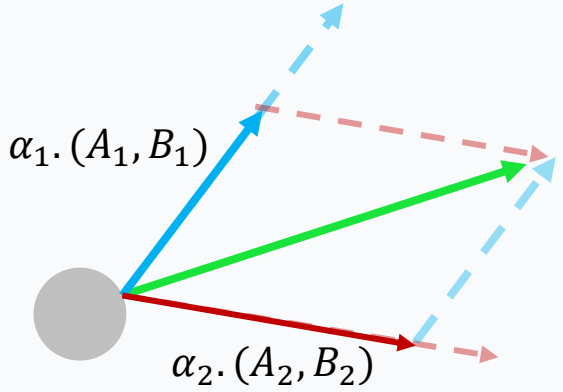
- **Remove Redundancy:** Keep only the **top k% largest parameters**.
- **Resolve sign conflicts:** Retain only parameters consistent with **the majority sign direction**.
- **Average:** Average only the **non-zero parameters**.

[1] Gabriel Ilharco et al. 2022. Editing models with task arithmetic. arXiv preprint arXiv:2212.04089 (2022).
[2] Prateek Yadav et al. 2023. Ties-merging: Resolving interference when merging models. Advances in Neural Information Processing Systems 36 (2023), 7093–7115.

2. Study Design

➢ Selected Model Fusion Approach

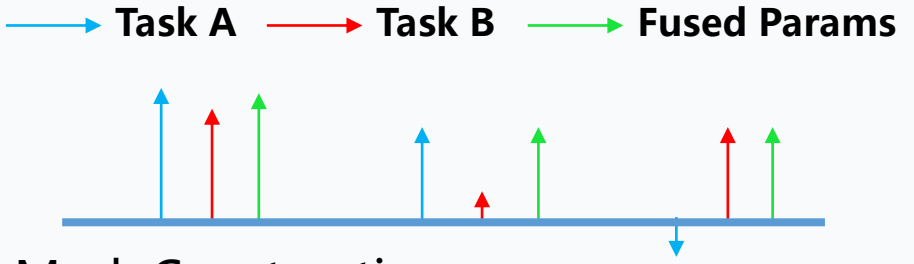
Learnable Fusion Coefficients[3]



$$\hat{A} = \sum_{i=0}^{N} \alpha_i \cdot A_i\,, \qquad \hat{B} = \sum_{i=0}^{N} \beta_i \cdot B_i$$

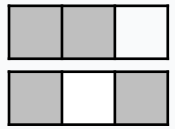
- Assign task-specific **fusion coefficients to different task vectors**.
- Optimize the fusion coefficients through **gradient descent**.

TALL-Masks [4]



- Mask Construction

$$m_i = 1\{|p_i| \geq |p_{fusion} - p_i|\}$$



If the **gap** between the fused parameters and task-specific parameters falls **within a certain range**, they are considered **related**.

- Inference

$$\hat{p_i} = m_i \odot p_{fusion}$$

[3] Enneng Yang et al. 2023. Adamerging: Adaptive model merging for multi-task learning. arXiv preprint arXiv:2310.02575 (2023).
[4] Ke Wang et al. Localizing task information for improved model merging and compression. arXiv preprint arXiv:2405.07813 (2024).

2. Study Design

➢ Experiment Settings

- Three program understand tasks (code search, vulnerability detection and clone detection)
- Three program generation tasks (code generation, code repair and code summarization)

Research questions (RQs)

RQ1: How does multi-PL fusion perform within the same task?

RQ2: How does fusion perform across multiple SE tasks?

RQ3: How does fusion perform across multiple tasks and multiple PLs?

RQ4: How can we improve the performance of model fusion techniques for SE tasks?

Dataset statistics for RQ1.

Task	Code Search			Code Repair	
	Train	Validation	Test	Train	Test
Python	412,178	23,107	22,176	461,356	710
Java	454,451	15,328	26,909	574,448	716
Javascript	123,889	8,253	6,483	8,861	183
Go	317,832	14,242	14,291	13,085	294
Ruby	48,791	2,209	2,279	5,153	343
PHP	523,712	26,015	28,391	3,595	191

Dataset statistics for RQ2 and RQ3.

	Language	Train	Validation	Test
Code Search	Java	412,178	23,107	22,176
Vulnerability Detection		70,008	7,825	7,135
Clone Detection		901,058	415,416	415,416
Code Generation	Python	50,661	\	164
Code Repair		461,356	\	710
Code Summarization		251,820	13,914	14,918

Normalized Accuracy (Metric for Model Fusion): $N.Acc. = \frac{S_{fusion}}{S_{ft}}$

3. Study Result

➢ RQ1: Performance of Multi-PL Fusion within Same Task

Task		Code Search							Code Repair						
		MRR (N. Acc.)							pass@5 (N. Acc.)						
Method	Learnable Coefficients	Python	Java	JavaScript	Go	Ruby	PHP	Avg.	Python	Java	JavaScript	Go	Ruby	PHP	Avg.
Fine-tuning	/	84.58	74.62	76.81	76.70	82.02	75.55	78.38	24.61	29.98	23.34	29.41	17.82	34.31	26.58
Task Arithmetic	no	82.68 (97.75)	74.84 (100.29)	77.74 (101.22)	75.19 (98.04)	81.57 (99.45)	72.41 (95.85)	77.41 (98.77)	21.72 (88.29)	27.18 (90.67)	27.28 (116.88)	24.89 (84.61)	20.08 (112.67)	27.22 (79.34)	24.73 (95.41)
	yes	82.10 (97.07)	74.67 (100.07)	77.69 (101.17)	74.69 (97.39)	81.45 (99.90)	72.82 (96.39)	77.24 (98.56)	23.08 (93.81)	28.17 (93.96)	**30.33 (129.94)**	26.17 (88.99)	**22.66 (127.12)**	**31.25 (91.09)**	**26.94 (104.15)**
Ties-Merging	no	82.40 (97.41)	74.99 (100.49)	77.56 (100.99)	73.67 (96.06)	80.96 (98.71)	73.79 (97.67)	77.23 (98.55)	23.77 (96.60)	29.20 (97.39)	26.69 (114.39)	17.3 (58.82)	20.02 (112.33)	20.94 (61.04)	22.99 (90.10)
	yes	82.52 (97.56)	75.08 (100.61)	**77.85 (101.37)**	73.83 (96.26)	81.54 (99.41)	73.74 (97.61)	77.43 (98.81)	23.88 (97.07)	28.71 (95.77)	26.99 (115.64)	19.75 (67.14)	20.83 (116.87)	23.08 (67.26)	23.87 (93.29)
TALL-Masks -max	/	**84.61 (100.03)**	**75.23 (100.84)**	77.48 (101.01)	**76.67 (99.97)**	**81.93 (99.88)**	**75.93 (100.49)**	**78.64 (100.37)**	**24.4 (99.14)**	**31.18 (103.99)**	23.33 (99.97)	**29.29 (99.56)**	17.71 (99.37)	26.69 (77.79)	25.43 (96.62)

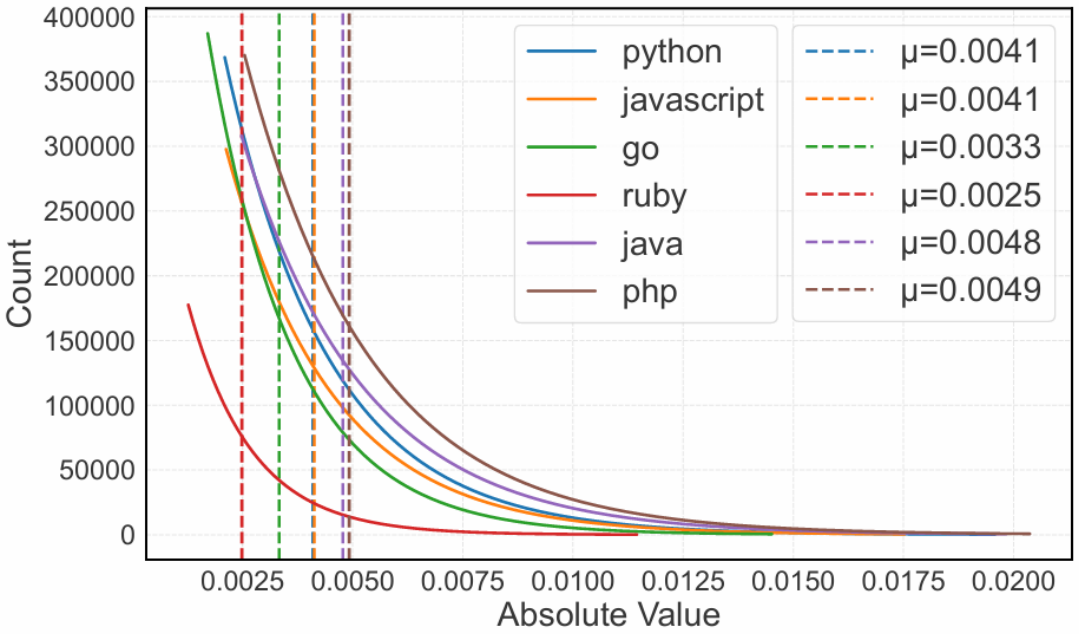
In code search, all methods maintain **relatively good performance**.

In code repair, all methods show a significant **performance drop on Go and PHP**.

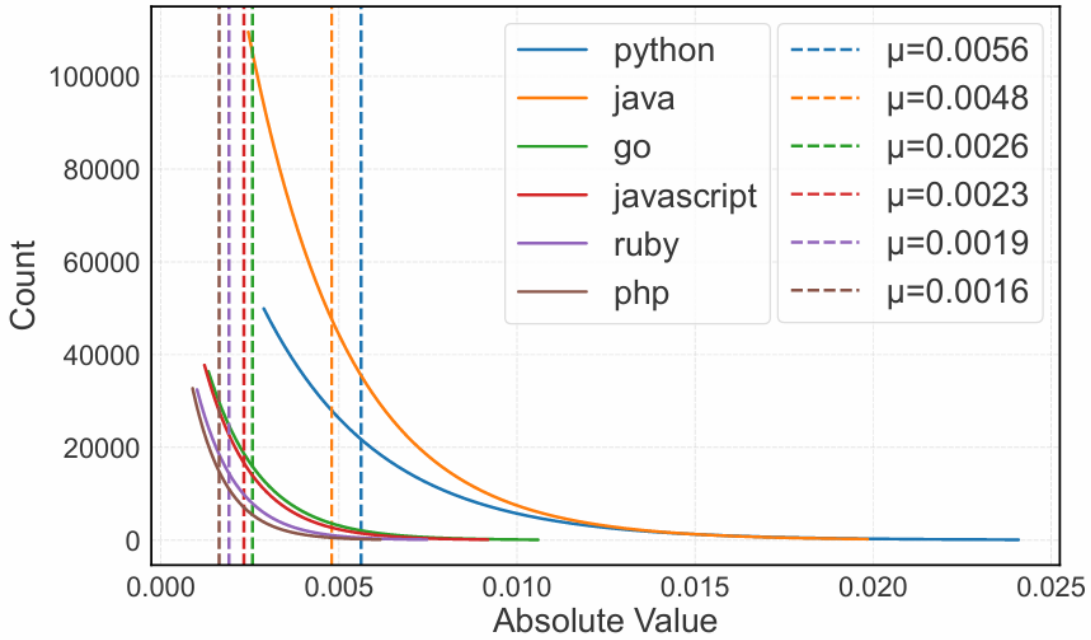
3. Study Result

➢ RQ1: Performance of Multi-PL Fusion within Same Task

Code Search



Code Repair



- For parameter distribution differences across PLs, code search shows much smaller variation than code repair.
- In code repair, **performance drops** usually occur on tasks **with smaller parameter changes**.

3. Study Result

➢ RQ1: Performance of Multi-PL Fusion within Same Task

Task		Code Search							Code Repair						
		MRR (N. Acc.)							pass@5 (N. Acc.)						
Method	Learnable Coefficients	Python	Java	JavaScript	Go	Ruby	PHP	Avg.	Python	Java	JavaScript	Go	Ruby	PHP	Avg.
Fine-tuning	/	84.58	74.62	76.81	76.70	82.02	75.55	78.38	24.61	29.98	23.34	29.41	17.82	34.31	26.58
Task Arithmetic	no	82.68 (97.75)	74.84 (100.29)	77.74 (101.22)	75.19 (98.04)	81.57 (99.45)	72.41 (95.85)	77.41 (98.77)	21.72 (88.29)	27.18 (90.67)	27.28 (116.88)	24.89 (84.61)	20.08 (112.67)	27.22 (79.34)	24.73 (95.41)
	yes	82.10 (97.07)	74.67 (100.07)	77.69 (101.17)	74.69 (97.39)	81.45 (99.90)	72.82 (96.39)	77.24 (98.56)	23.08 (93.81)	28.17 (93.96)	**30.33 (129.94)**	26.17 (88.99)	**22.66 (127.12)**	**31.25 (91.09)**	**26.94 (104.15)**
Ties-Merging	no	82.40 (97.41)	74.99 (100.49)	77.56 (100.99)	73.67 (96.06)	80.96 (98.71)	73.79 (97.67)	77.23 (98.55)	23.77 (96.60)	29.20 (97.39)	26.69 (114.39)	17.3 (58.82)	20.02 (112.33)	20.94 (61.04)	22.99 (90.10)
	yes	82.52 (97.56)	75.08 (100.61)	**77.85 (101.37)**	73.83 (96.26)	81.54 (99.41)	73.74 (97.61)	77.43 (98.81)	23.88 (97.07)	28.71 (95.77)	26.99 (115.64)	19.75 (67.14)	20.83 (116.87)	23.08 (67.26)	23.87 (93.29)
TALL-Masks -max	/	**84.61 (100.03)**	**75.23 (100.84)**	77.48 (101.01)	**76.67 (99.97)**	**81.93 (99.88)**	**75.93 (100.49)**	**78.64 (100.37)**	**24.4 (99.14)**	**31.18 (103.99)**	23.33 (99.97)	**29.29 (99.56)**	17.71 (99.37)	26.69 (77.79)	25.43 (96.62)

Performance improves on certain PLs.

3. Study Result

➢ RQ1: Performance of Multi-PL Fusion within Same Task

```
// The Code of JavaScript
function main() {
  //...

  const weight = readline();
- // buggy
- const output = weight % 2 === 0 ? "YES" : "NO"

+ // fixed after fused model
+ const output = (weight % 2 === 0 && weight > 2)
      ? "YES" : "NO"
  //...
}
```

```
// The Code of Go
func main() {
  //...
  fmt.Scan(&a)
- // buggy
- if a % 2 == 0 {
+ // fixed by fine-tuned models
+ if a % 2 == 0 && a > 2 {
    fmt.Println("YES")
  } else {
    fmt.Println("NO")
  }
}
```

In the code repair task, when bugs share similarities, fusion can improve performance.

3. Study Result

➢ RQ2: Performance of Multi-SE-Task Fusion within Same PL

Fusion Performance of Program Understanding Tasks (Java)

Task		Code Search	Vulnerability Detection				Clone Detection			
Method	Learnable Coefficients	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)
Fine-tuning	/	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22
Task Arithmetic	no	68.87 (92.29)	4.14	16.18	**46.45**	24.00 (80.00)	98.43	96.88	91.52	94.12 (98.72)
	yes	68.34 (91.58)	90.84	35.53	26.11	30.11 (98.11)	98.69	**97.79**	92.57	95.11 (99.88)
Ties-Merging	no	69.71 (93.41)	0.00*	0.00	0.00	0.00 (0.00)	98.62	95.17	**94.73**	94.95 (99.71)
	yes	64.06 (85.85)	90.75	19.61	5.60	8.71 (28.38)	98.17	97.41	89.01	93.02 (97.68)
TALL-Masks	/	**74.75 (100.17)**	**92.55**	**51.06**	22.39	**31.12 (101.40)**	**98.71**	96.47	94.01	**95.23 (100.01)**

Fusion Performance of Program Generation Tasks (Python)

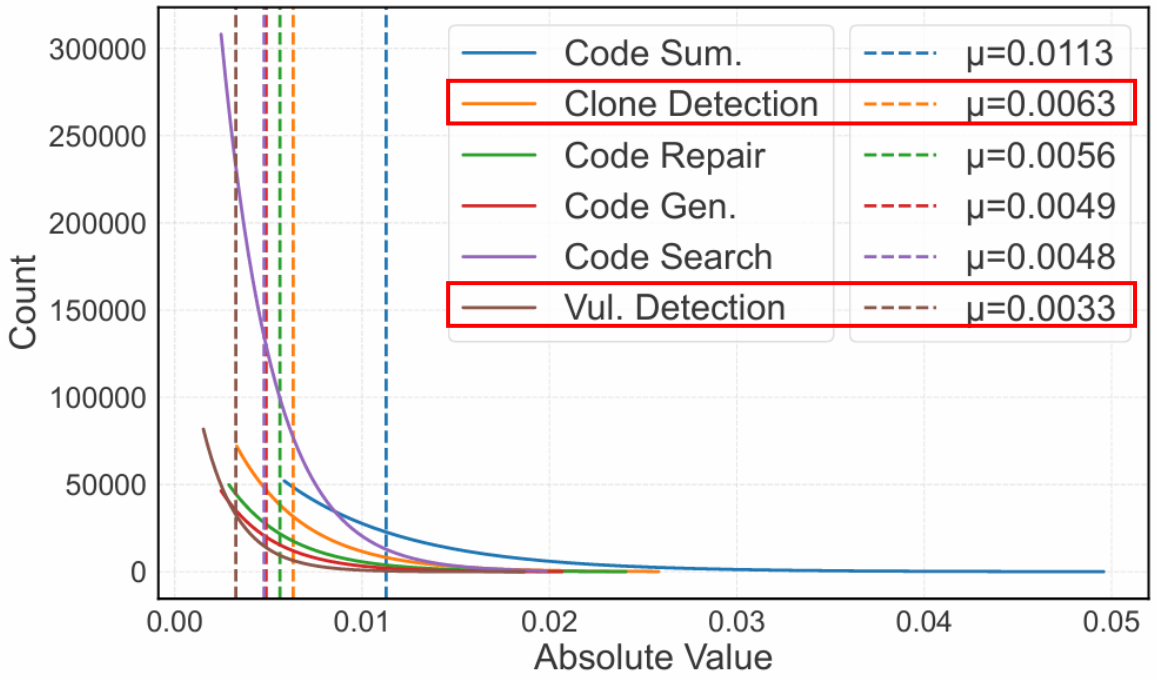
Task		Code Repair	Code Sum.		Code Gen.
Method	Learnable Coefficients	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	/	24.61	8.13	88.66	67.00
Task Arithmetic	no	17.55 (71.31)	8.76 (107.75)	88.72 (100.07)	61.70 (92.90)
	yes	18.89 (76.75)	8.86 (108.98)	88.90 (100.07)	53.60 (80.00)
Ties-Merging	no	10.54 (42.82)	**9.65 (118.69)**	**89.13 (100.53)**	57.70 (86.12)
	yes	17.09 (69.44)	9.05 (113.16)	88.92 (100.29)	55.80 (83.28)
TALL-Masks	/	**23.59 (95.86)**	8.30 (102.09)	88.72 (100.07)	**66.80 (99.70)**

Except for TALL-Masks, other methods suffer **significant performance losses on certain tasks**.

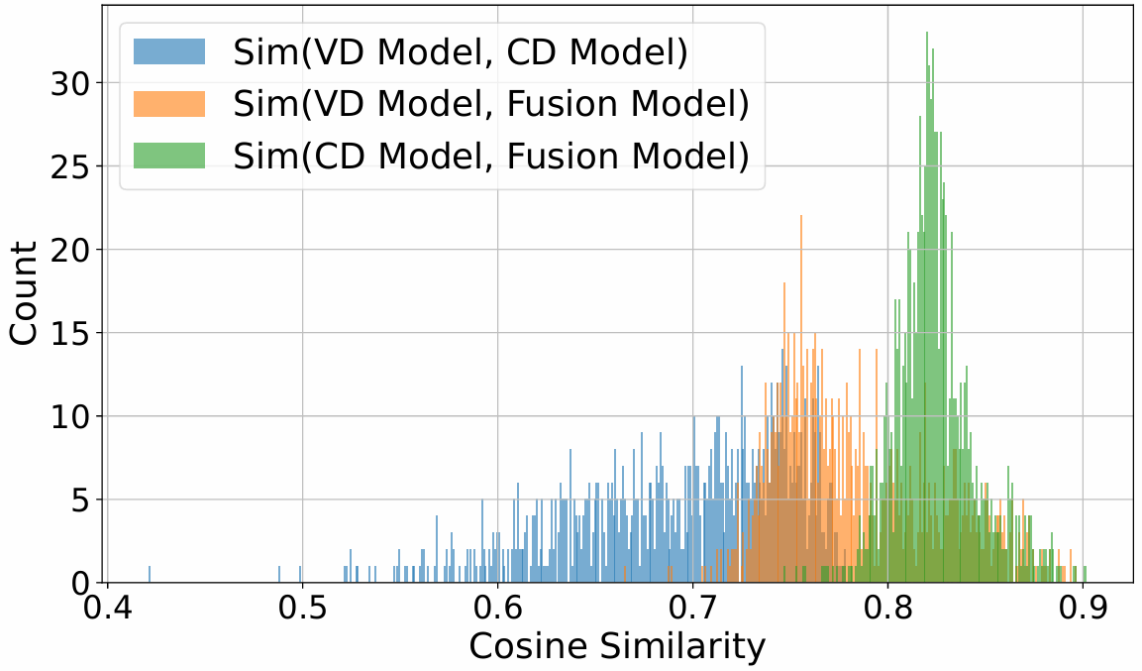
All methods can maintain relatively **good performance on certain tasks**.

3. Study Result

➢ RQ2: Performance of Multi-SE-Task Fusion within Same PL



Parameter change distributions vary greatly across different tasks.



The fused model **behaves more similarly** to fine-tuned models **from tasks with stronger features** (larger parameter changes).

3. Study Result

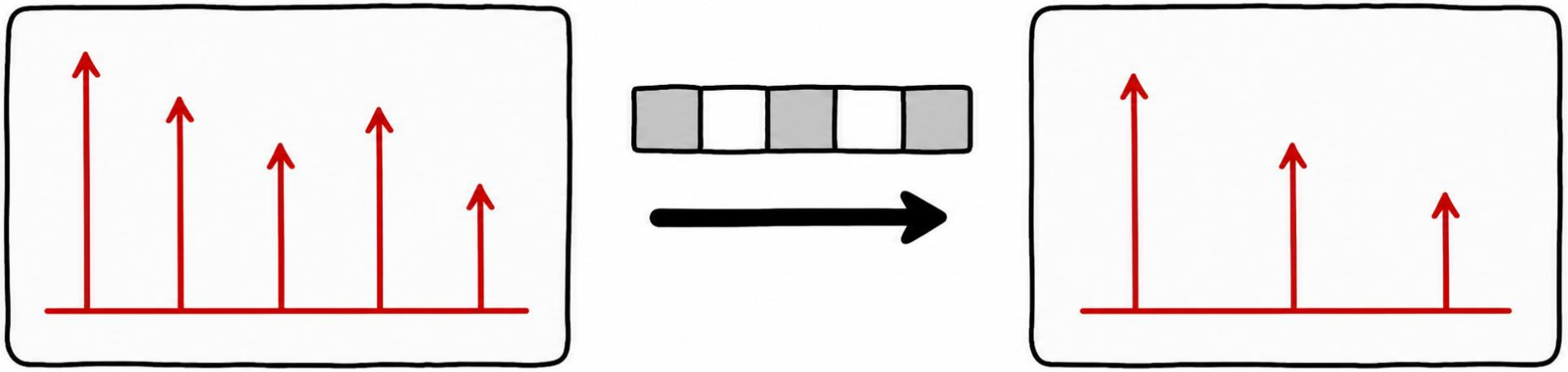
➢ RQ3: Performance of Multi-SE-Task Multi-PL Fusion

Task		Code Search	Vulnerability Detection				Clone Detection				Code Repair	Code Sum.		Code Gen.
Method	Learnable Coefficients	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	/	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22	24.61	8.13	88.66	67.00
Task Arithmetic	no	68.06 (91.21)	78.41	17.31	49.62	**25.67 (83.64)**	94.93	**99.75**	63.09	77.29 (81.16)	15.38 (62.49)	1.11 (13.65)	80.85 (91.19)	60.30 (90.00)
	yes	62.24 (83.41)	**91.46**	28.15	8.95	13.67 (44.54)	98.42	99.14	89.23	93.92 (98.63)	15.08 (61.27)	**8.56 (105.29)**	**88.93 (100.30)**	53.90 (80.45)
Ties-Merging	no	66.68 (89.36)	26.52	9.27	**83.39**	16.70 (54.41)	94.99	97.27	65.19	78.06 (81.98)	10.91 (44.33)	1.25 (15.37)	81.76 (92.22)	56.40 (84.18)
	yes	66.57 (89.21)	91.17	9.52	1.11	2.03 (6.61)	89.98	76.65	38.47	51.23 (53.80)	13.86 (56.31)	6.98 (84.75)	88.38 (99.68)	46.40 (69.25)
TALL-Masks	/	**73.78 (98.87)**	58.24	**77.27**	6.34	11.72 (36.72)	**98.81**	97.17	**94.02**	**95.57 (100.37)**	**24.23 (98.45)**	**8.56 (105.29)**	88.75 (100.10)	**68.10 (101.64)**

- **Almost all fusion methods** experience **significant performance drops** on certain tasks.
- **TALL-Masks suffers a notable performance drop on vulnerability detection**, while retaining nearly 100% of the fine-tuned model’ s capability on other tasks.

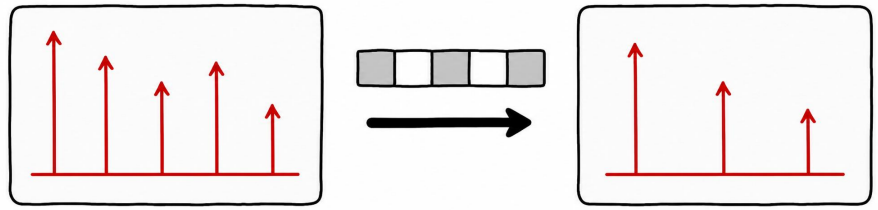
3. Study Result

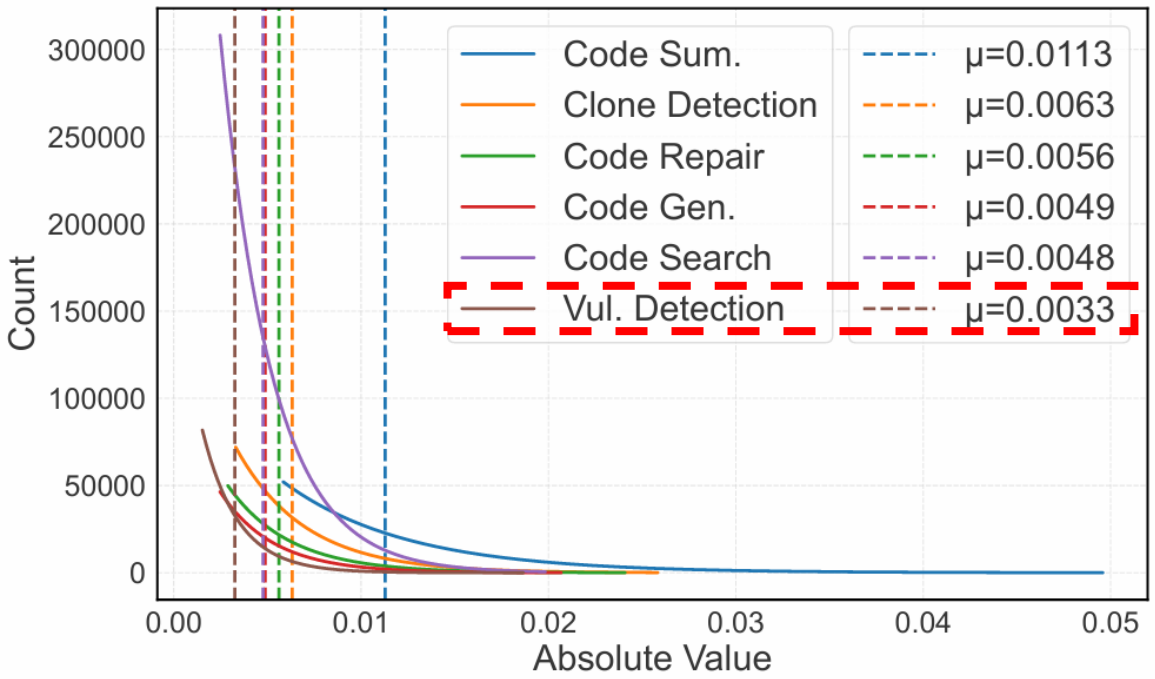
- **RQ3: Performance of Multi-SE-Task Multi-PL Fusion**

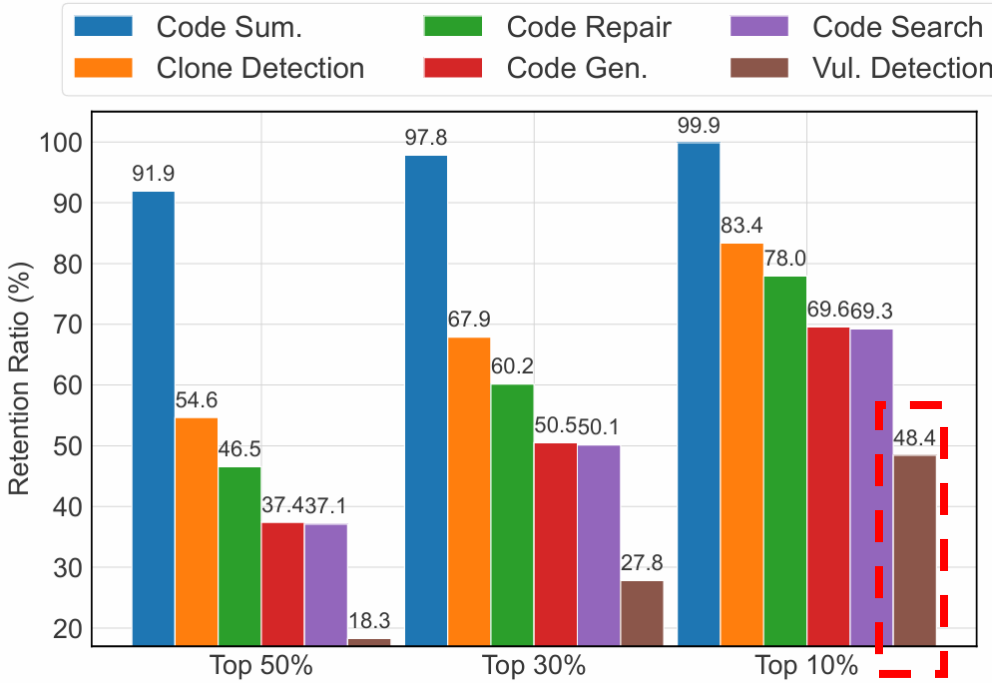


3. Study Result

➢ RQ3: Performance of Multi-SE-Task Multi-PL Fusion



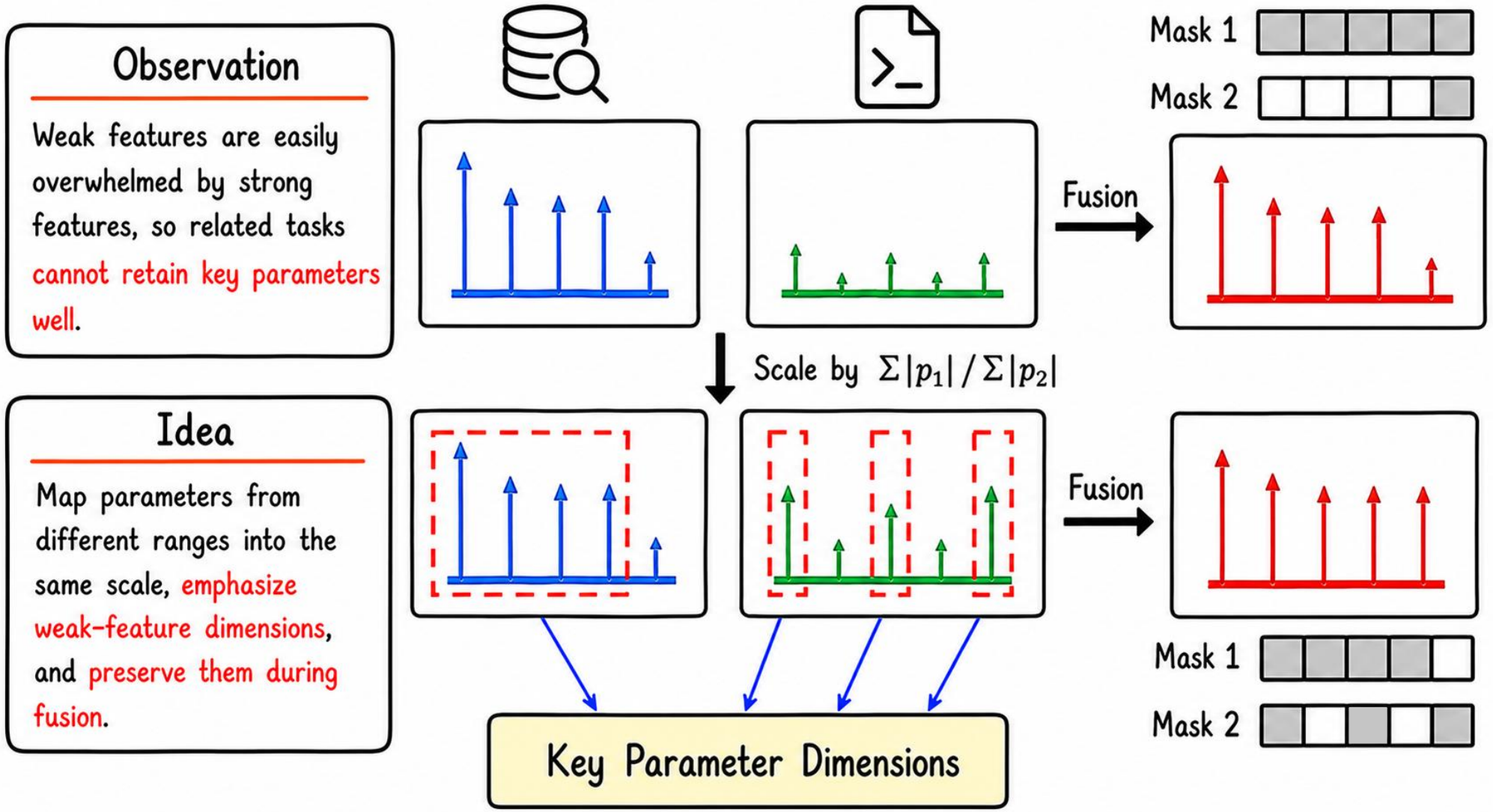




For TALL-Masks, **on weak-feature tasks**, the constructed **masks fail to preserve important parameters** effectively .

3. Study Result

➢ RQ4: Enhancement Strategy (Scaling-Masks)



3. Study Result

➢ RQ4: Enhancement Strategy (Scaling-Masks)

Performance of Multi-PL Fusion within Same Task

Task	Code Search							Code Repair						
	MRR (N. Acc.)							pass@5 (N. Acc.)						
Method	Python	Java	JavaScript	Go	Ruby	PHP	Avg.	Python	Java	JavaScript	Go	Ruby	PHP	Avg.
Fine-tuning	84.58	74.62	76.81	76.70	82.02	75.55	78.38	24.61	29.98	23.34	29.41	17.82	34.31	26.58
TALL-Masks	84.61 (100.03)	75.23 (100.84)	**77.48 (101.01)**	76.67 (99.97)	81.93 (99.88)	75.93 (100.49)	78.64 (100.37)	24.4 (99.14)	31.18 (103.99)	23.33 (99.97)	29.29 (99.56)	17.71 (99.37)	26.69 (77.79)	25.43 (96.62)
Scaling-Masks	**84.63 (100.05)**	**75.28 (100.88)**	**77.48 (101.01)**	**76.74 (100.05)**	**82.09 (100.08)**	**75.95 (100.53)**	**78.70 (100.41)**	**25.02 (101.67)**	**31.57 (105.30)**	**24.36 (104.37)**	**30.71 (104.42)**	**18.77 (105.33)**	**34.93 (101.81)**	**27.56 (103.81)**

Performance of Multi-SE-Task Multi-PL Fusion

Task	Code Search	Vulnerability Detection				Clone Detection				Code Repair	Code Sum.		Code Gen.
Method	MRR (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	Acc.	Prec.	Rec.	F1 (N. Acc.)	pass@5 (N. Acc.)	BLEU (N. Acc.)	bert-score (N. Acc.)	pass@5 (N. Acc.)
Fine-tuning	74.62	92.44	50.21	22.1	30.69	98.71	96.24	94.23	95.22	24.61	8.13	88.66	67.00
TALL-Masks	73.78 (98.87)	58.24	**77.27**	6.34	11.72 (36.72)	**98.81**	**97.17**	94.02	**95.57 (100.37)**	24.23 (98.45)	8.56 (105.29)	88.75 (100.10)	**68.10 (101.64)**
Scaling-Masks	**74.63 (100.01)**	**92.42**	48.98	**22.57**	**30.90 (100.64)**	98.79	96.9	**94.2**	95.54 (100.33)	**24.88 (101.10)**	**9.03 (111.07)**	**88.90 (100.27)**	67.20 (100.30)

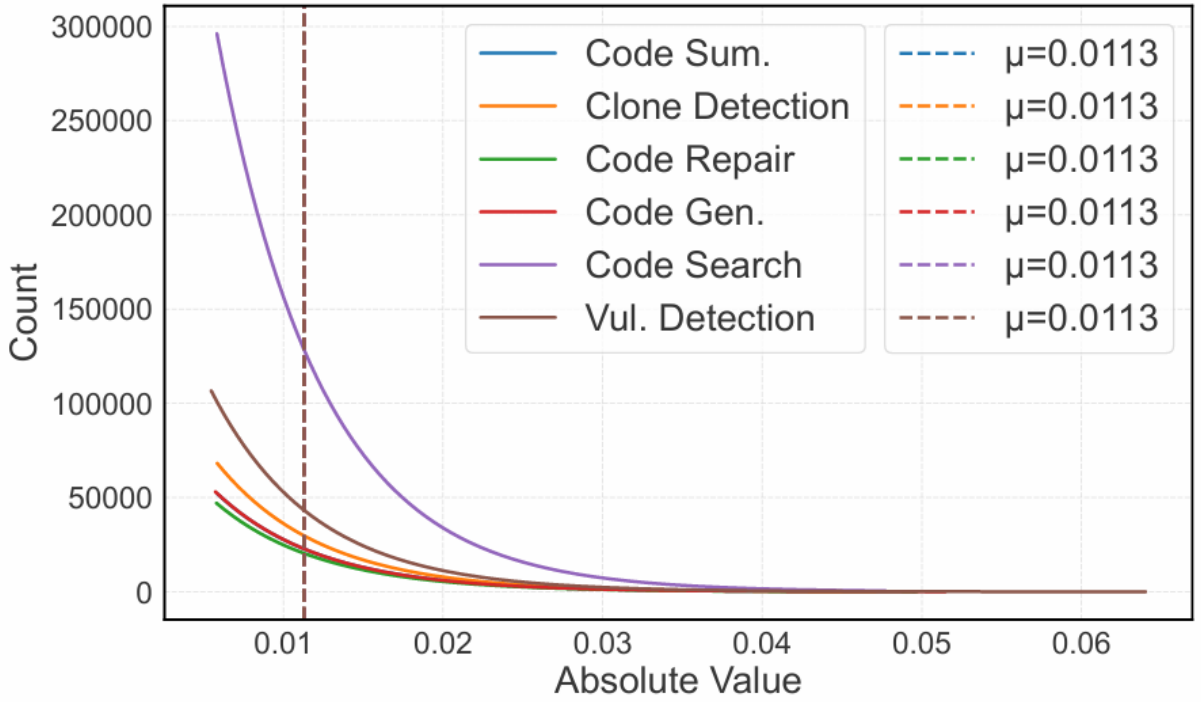
Scaling-Masks retains **100%** of the original fine-tuned model's performance **across all tasks**.

24.0% PHP

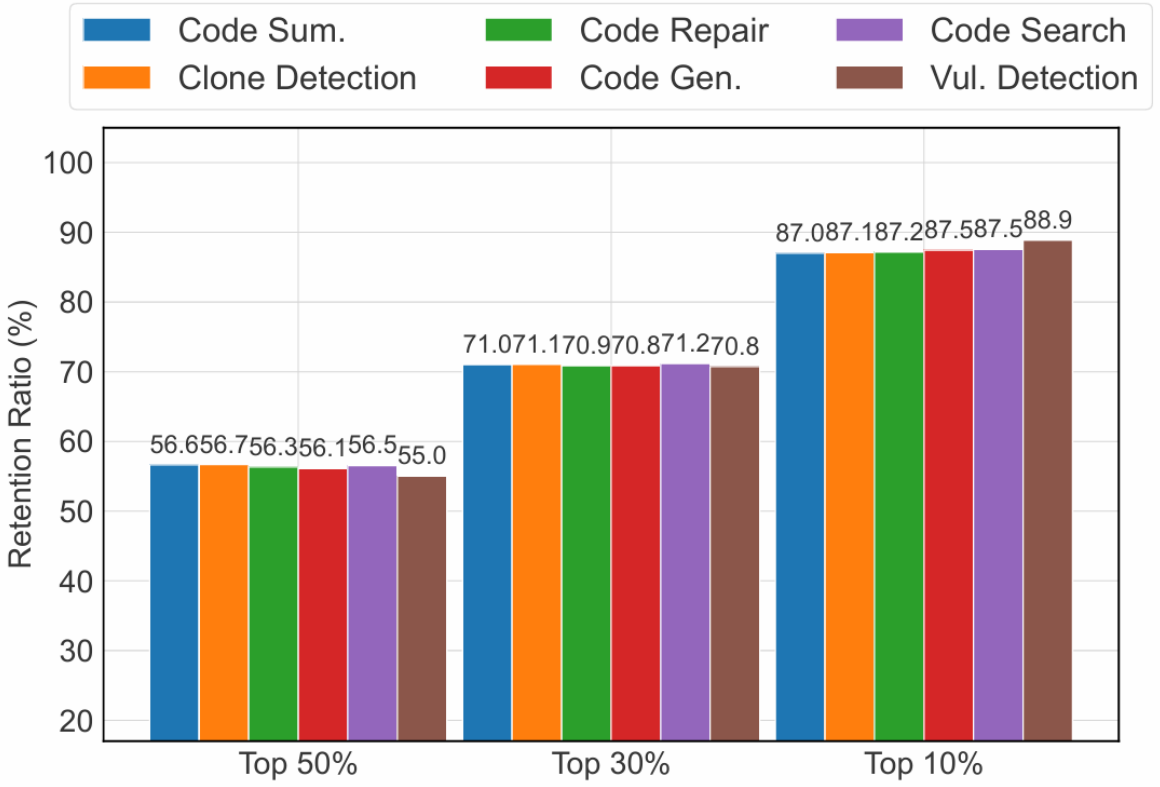
63.4% Vul. Detection

3. Study Result

➢ RQ4: Enhancement Strategy (Scaling-Masks)



The normalized parameter **distributions is more similar**.



Key parameter **retention rate is significantly improved**.

Q & A

Please email qyg@nudt.edu.cn with your questions

The source code of this work is available at
https://doi.org/10.5281/zenodo.19491719

